

Python 程式語言

Lecture 3: Data Structures and Collections

Instructor: 顏家珩 Chia-Heng Yen

Course Roadmap

- Fundamentals & Environment
 - Introduction to Python and Development Environment Setup
 - Basic Syntax and Flow Control
 - **Data Structures and Collections**
- Advanced Programming
 - Modular and Functional Programming
 - File Handling and Exception Handling
 - Object-Oriented Programming
 - Algorithm Implementation in Python
- User Interface Development
 - GUI Programming with Tkinter and PyQt
- Data Processing & Analysis
 - Data Analysis Fundamentals
 - Scientific Computing Applications
 - Data Visualization
- AI & ML Applications
 - Introduction to Machine Learning with Applications
 - AI-Related Library in Python Applications

Week 2 Recap

- Master Python syntax fundamentals
- Understand data types and operations
- Implement decision-making logic
- Create repetitive task automation

Outline

- Master Python built-in **data structures**
- Choose appropriate **data structures** for tasks
- Implement efficient data manipulation
- Apply **collections** in real-world scenarios

Why Data Structures Matter?

- Example 1: Student Management

```
# Without proper data structures
```

```
student1_name = "Alice"
```

```
student1_grade = 85
```

```
student1_courses = "Math,Physics,English"
```

```
student2_name = "Bob"
```

```
student2_grade = 92
```

```
student2_courses = "Chemistry,Biology,Math"
```

This becomes unmanageable quickly!

Why Data Structures Matter?

- Example 1: Student Management

```
# With proper data structures
students = [
    {
        "name": "Alice",
        "grade": 85,
        "courses": ["Math", "Physics", "English"]
    },
    {
        "name": "Bob",
        "grade": 92,
        "courses": ["Chemistry", "Biology", "Math"]
    }
]
```

Much more organized and scalable!

Why Data Structures Matter?

- Example 2: E-commerce Shopping Cart

Poor approach

```
item1 = "iPhone"  
quantity1 = 2  
price1 = 999
```

```
item2 = "AirPods"  
quantity2 = 1  
price2 = 249
```

Better approach

```
shopping_cart = {  
    "iPhone": {"quantity": 2, "price": 999},  
    "AirPods": {"quantity": 1, "price": 249}  
}
```

```
total = sum(item["quantity"] * item["price"]  
            for item in shopping_cart.values())
```

The Foundation of Programming

- Benefits of Proper Data Structures
 - Efficiency: Faster data access and manipulation
 - Scalability: Handle growing amounts of data
 - Readability: Code is easier to understand
 - Maintainability: Easier to modify and extend
 - Performance: Optimize memory usage and speed
- Choosing the Right Structure
 - **Lists** for ordered, changeable collections
 - **Tuples** for unchangeable, structured data
 - **Sets** for unique items and fast lookups
 - **Dictionaries** for key-value relationships

Python Collection Types Overview

- Collection Characteristics Matrix

Type	Ordered (有序)	Mutable (可變)	Duplicates (重複)	Indexed (索引)
List (串列)	Yes	Yes	Yes	Yes
Tuple (多元組)	Yes	No	Yes	Yes
Set (集合)	No	Yes	No	No
Dict (字典)	Yes*	Yes	No**	Keys

*Since Python 3.7+

**Keys must be unique, values can duplicate

When to Use Each Type

- Lists - **The Workhorses**

Use for: ordered collections that change

```
grades = [85, 92, 78, 96, 89]
```

```
students = ["Alice", "Bob", "Charlie"]
```

```
mixed_data = [1, "hello", 3.14, True]
```

Common operations

```
grades.append(95)
```

Add item

```
grades.sort()
```

Sort in place

```
best_grade = max(grades)
```

Find maximum

When to Use Each Type

- Tuples - **Immutable Containers**

Use for: fixed collections, coordinates, config

```
point = (10, 20)
```

```
rgb_color = (255, 128, 0)
```

```
database_config = ("localhost", 5432, "mydb")
```

Common operations

```
x, y = point
```

Unpacking

```
count = point.count(10)
```

Count occurrences

When to Use Each Type

- Sets - **Unique Collections**

Use for: unique items, membership testing

```
unique_visitors = {"alice", "bob", "charlie"}
```

```
technologies = {"python", "javascript", "sql"}
```

Common operations

```
unique_visitors.add("diana")
```

Add member

```
common_tech = technologies & {"python", "java"}
```

Intersection

When to Use Each Type

- Dictionaries - **Key-Value Mappings**

Use for: lookups, structured data

```
student = {  
    "name": "Alice",  
    "age": 20,  
    "courses": ["Math", "Physics"]  
}
```

Common operations

```
student["gpa"] = 3.8           # Add/update  
courses = student.get("courses", [])  # Safe access
```

Type Conversion Between Collections

Converting between types

```
numbers_list = [1, 2, 3, 2, 1]
```

```
numbers_tuple = tuple(numbers_list)      # (1, 2, 3, 2, 1)
```

```
numbers_set = set(numbers_list)          # {1, 2, 3} - duplicates removed
```

```
numbers_back = list(numbers_set)         # [1, 2, 3] - order may change
```

Creating from strings

```
word = "hello"
```

```
letters_list = list(word)               # ['h', 'e', 'l', 'l', 'o']
```

```
unique_letters = set(word)              # {'h', 'e', 'l', 'o'}
```

Performance Considerations

```
# Time complexity for common operations
# Access by Index: List/Tuple  $O(1)$ , Set/Dict N/A
# Search: List  $O(n)$ , Set/Dict  $O(1)$  average
# Insert: List  $O(1)$  at end,  $O(n)$  in middle; Set/Dict  $O(1)$  average
# Delete: List  $O(n)$ , Set/Dict  $O(1)$  average
```

```
# Example: Fast membership testing
```

```
large_list = list(range(1000000))
```

```
large_set = set(large_list)
```

```
# This is slow
```

```
# if 999999 in large_list: #  $O(n)$  - checks every element
```

```
# This is fast
```

```
if 999999 in large_set: #  $O(1)$  - direct lookup
    print("Found!")
```

Understanding Python Memory

- Mutable vs Immutable Objects
 - Immutable Types: Numbers, Strings, Tuples, Frozensets

```
# Immutable example
```

```
text = "Hello"
```

```
print(id(text))           # Memory address
```

```
text = text + " World"    # Creates NEW object
```

```
print(id(text))           # Different address!
```

```
# Original "Hello" still exists in memory until garbage collected
```


Understanding Python Memory

- Mutable vs Immutable Objects
 - Mutable Types: Lists, Sets, Dictionaries

```
# Mutable example  
numbers = [1, 2, 3]  
print(id(numbers))           # Memory address  
  
numbers.append(4)             # Modifies SAME object  
print(id(numbers))           # Same address!  
print(numbers)                # [1, 2, 3, 4]
```

Understanding Python Memory

- Assignment vs Copying
 - Assignment Creates References

```
# Reference assignment
original_list = [1, 2, 3]
reference_list = original_list          # Same object!
reference_list.append(4)
print(original_list)                   # [1, 2, 3, 4] - Both changed!
print(reference_list)                 # [1, 2, 3, 4]

# Check if they're the same object
print(original_list is reference_list) # True
```

Understanding Python Memory

- Assignment vs Copying
 - Shallow Copy

```
# Create new object with same contents
original = [1, 2, 3]
shallow_copy = original.copy()           # or list(original) or original[:]

shallow_copy.append(4)
print(original)                         # [1, 2, 3] - Unchanged!
print(shallow_copy)                     # [1, 2, 3, 4]
```

Understanding Python Memory

- Assignment vs Copying
 - Deep Copy for Nested Objects

```
import copy

# Nested list example
original = [[1, 2], [3, 4]]
shallow = original.copy()
deep = copy.deepcopy(original)

# Modify nested element
original[0].append(3)

print(original) # [[1, 2, 3], [3, 4]]
print(shallow) # [[1, 2, 3], [3, 4]] - Also changed!
print(deep) # [[1, 2], [3, 4]] - Unchanged!
```

Practical Implications

- Function Parameters

```
def modify_list(lst):
    """This modifies the original list!"""
    lst.append("modified")
    return lst

def modify_string(text):
    """This creates a new string"""
    text = text + " modified"
    return text

# Test with list
my_list = [1, 2, 3]
result = modify_list(my_list)
print(my_list) # [1, 2, 3, 'modified'] - Original changed!

# Test with string | 使用字串測試
my_string = "Hello"
result = modify_string(my_string)
print(my_string) # "Hello" - Original unchanged!
```

Practical Implications

- Safe List Operations

```
def safe_append(lst, item):  
    """Return new list without modifying original"""  
    return lst + [item]          # Creates new list  
  
def safe_modify(lst):  
    """Work with copy to avoid side effects"""  
    local_copy = lst.copy()  
    local_copy.append("new item")  
    return local_copy  
  
# Usage  
original = [1, 2, 3]  
new_list = safe_append(original, 4)  
print(original)                # [1, 2, 3] - Unchanged  
print(new_list)                 # [1, 2, 3, 4]
```

Practical Implications

- Memory Efficiency Tips

Efficient string building
Bad: Creates many intermediate strings

```
result = ""  
for i in range(1000):  
    result += str(i) + " "
```

Good: Use join for multiple concatenations

```
numbers = [str(i) for i in range(1000)]  
result = " ".join(numbers)
```

List pre-allocation when size known

Instead of growing list

```
squares = []  
for i in range(1000):  
    squares.append(i * i)
```

Pre-allocate if possible

```
squares = [0] * 1000  
for i in range(1000):  
    squares[i] = i * i
```

Or use list comprehension

```
squares = [i * i for i in range(1000)]
```



List



List Basics

- What are Lists?
 - Lists are ordered, mutable collections that can store any type of data
- Key Characteristics
 - Ordered (有序): Items maintain their position
 - Mutable (可變): Can be modified after creation
 - Allow duplicates (允許重複): Same values can appear multiple times
 - Heterogeneous (異質): Can contain different data types

List Creation

- Creation Methods

Method 1: Square brackets

```
fruits = ["apple", "banana", "orange"]
```

```
numbers = [1, 2, 3, 4, 5]
```

```
mixed = ["Python", 42, 3.14, True]
```

Method 2: list() constructor

```
numbers = list(range(1, 6)) # [1, 2, 3, 4, 5]
```

```
letters = list("Python") # ['P', 'y', 't', 'h', 'o', 'n']
```

Method 3: List repetition

```
zeros = [0] * 5 # [0, 0, 0, 0, 0]
```

```
pattern = [1, 2] * 3 # [1, 2, 1, 2, 1, 2]
```

Method 4: List comprehension

```
squares = [x**2 for x in range(5)] # [0, 1, 4, 9, 16]
```

Accessing List Elements

- Indexing System

```
fruits = ["apple", "banana", "orange", "grape", "kiwi"]  
#           0           1           2           3           4   (positive index)  
#          -5          -4          -3          -2          -1   (negative index)
```

- Positive Indexing

```
first_fruit = fruits[0]   # "apple"  
second_fruit = fruits[1] # "banana"  
last_fruit = fruits[4]   # "kiwi"
```

- Negative Indexing

```
last_fruit = fruits[-1]  # "kiwi" (last item)  
second_last = fruits[-2] # "grape" (second to last)  
first_fruit = fruits[-5] # "apple" (same as fruits[0])
```

Accessing List Elements

- Index Error Handling

```
# This will cause IndexError  
# invalid_item = fruits[10]  
  
# Safe access  
def safe_get(lst, index, default=None):  
    if 0 <= index < len(lst) or -len(lst) <= index < 0:  
        return lst[index]  
    return default  
  
item = safe_get(fruits, 10, "Not found") # "Not found"
```

List Slicing - Extract Sublists

- Slicing Syntax

```
list[start:stop:step]
# start: inclusive
# stop: exclusive
# step: default is 1
numbers = [0, 1, 2, 3, 4, 5, 6, 7, 8, 9]
```

- Basic Slicing

```
first_five = numbers[0:5]           # [0, 1, 2, 3, 4]
middle = numbers[3:7]               # [3, 4, 5, 6]
from_start = numbers[:5]            # [0, 1, 2, 3, 4]
to_end = numbers[5:]                # [5, 6, 7, 8, 9]
everything = numbers[:]              # [0, 1, 2, 3, 4, 5, 6, 7, 8, 9] (copy)
```

List Slicing - Extract Sublists

- Slicing Syntax

```
list[start:stop:step]
# start: inclusive
# stop: exclusive
# step: default is 1
numbers = [0, 1, 2, 3, 4, 5, 6, 7, 8, 9]
```

- Advanced Slicing

```
every_second = numbers[::2]           # [0, 2, 4, 6, 8]
every_third = numbers[1::3]           # [1, 4, 7]
reversed_all = numbers[::-1]          # [9, 8, 7, 6, 5, 4, 3, 2, 1, 0]
last_three = numbers[-3:]             # [7, 8, 9]
```

List Slicing - Extract Sublists

- Slicing Syntax

```
list[start:stop:step]
# start: inclusive
# stop: exclusive
# step: default is 1
numbers = [0, 1, 2, 3, 4, 5, 6, 7, 8, 9]
```

- Modifying with Slicing

```
numbers[1:4] = [10, 20, 30] # Replace elements
numbers[2:2] = [100] # Insert elements
del numbers[1:3] # Delete elements
```

Essential List Methods

- Adding Elements

append() - Add single item to end

```
fruits = ["apple", "banana"]
```

```
fruits.append("orange")           # ['apple', 'banana', 'orange']
```

insert() - Add at specific position

```
fruits.insert(1, "grape")         # ['apple', 'grape', 'banana', 'orange']
```

extend() - Add multiple items

```
fruits.extend(["kiwi", "mango"])  # ['apple', 'grape', 'banana', 'orange',  
                                   'kiwi', 'mango']
```

+ operator - Create new list

```
new_fruits = fruits + ["pear"]
```


Essential List Methods

- Removing Elements

remove() - Remove first occurrence by value

`fruits.remove("banana")` *# Removes first "banana"*

pop() - Remove by index and return value

`last_item = fruits.pop()` *# Remove and return last item*

`second_item = fruits.pop(1)` *# Remove and return item at index 1*

clear() - Remove all elements

`fruits.clear()` *# []*

del statement - Remove by index or slice

`numbers = [1, 2, 3, 4, 5]`

`del numbers[0]` *# Remove first element*

`del numbers[1:3]` *# Remove slice*

Finding and Organizing Elements

- Searching Methods

```
fruits = ["apple", "banana", "orange", "banana", "grape"]
```

index() - Find position of first occurrence

```
banana_pos = fruits.index("banana")           # 1
```

```
orange_pos = fruits.index("orange")           # 2
```

count() - Count occurrences

```
banana_count = fruits.count("banana")         # 2
```

```
apple_count = fruits.count("apple")           # 1
```

in/not in - Check membership

```
if "apple" in fruits:  
    print("Apple found!")
```

```
if "mango" not in fruits:  
    print("No mango!")
```

Finding and Organizing Elements

- Sorting Methods

```
numbers = [3, 1, 4, 1, 5, 9, 2, 6]
```

```
# sort() - Sort in place (modifies original)
```

```
numbers.sort() # [1, 1, 2, 3, 4, 5, 6, 9]
```

```
numbers.sort(reverse=True) # [9, 6, 5, 4, 3, 2, 1, 1]
```

```
# sorted() - Return new sorted list
```

```
original = [3, 1, 4, 1, 5]
```

```
sorted_list = sorted(original)
```

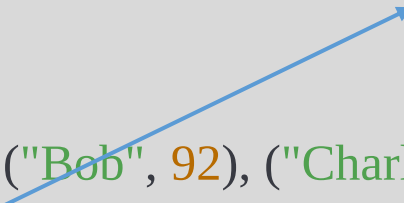
```
# original: [3, 1, 4, 1, 5] (unchanged)
```

```
# sorted_list: [1, 1, 3, 4, 5]
```

```
# Custom sorting
```

```
students = [("Alice", 85), ("Bob", 92), ("Charlie", 78)]
```

```
students.sort(key=lambda x: x[1]) # Sort by grade
```



```
def get_grade(student_tuple):  
    return student_tuple[1]  
key=get_grade
```

Creating Lists Efficiently

- Basic Syntax

```
# [expression for item in iterable]
```

```
# Traditional way
```

```
squares = []
```

```
for x in range(10):
```

```
    squares.append(x**2)
```

```
# List comprehension (推導式)
```

```
squares = [x**2 for x in range(10)] # Much cleaner!
```

- Performance Benefit
 - List comprehensions are faster than equivalent for loops

Creating Lists Efficiently

- Basic Examples

Generate numbers

```
numbers = [i for i in range(1, 6)] # [1, 2, 3, 4, 5]
```

Transform strings

```
names = ["alice", "bob", "charlie"]
```

```
upper_names = [name.upper() for name in names] # ['ALICE', 'BOB', 'CHARLIE']
```

Mathematical operations

```
celsius = [0, 20, 30, 40]
```

```
fahrenheit = [c * 9/5 + 32 for c in celsius] # [32.0, 68.0, 86.0, 104.0]
```

Extract from objects

```
students = [("Alice", 85), ("Bob", 92)]
```

```
names = [name for name, grade in students] # ['Alice', 'Bob']
```

```
grades = [grade for name, grade in students] # [85, 92]
```

Complex List Comprehensions

- With Conditions

Filter with if clause

```
evens = [x for x in range(20) if x % 2 == 0] # [0, 2, 4, 6, 8, 10, 12, 14, 16, 18]
```

Multiple conditions

```
numbers = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
```

```
special = [x for x in numbers if x > 3 and x < 8] # [4, 5, 6, 7]
```

Conditional expression (ternary)

```
result = ["even" if x % 2 == 0 else "odd" for x in range(5)]
```

```
# ['even', 'odd', 'even', 'odd', 'even']
```

Complex List Comprehensions

- Nested Comprehensions

Nested loops

```
pairs = [(i, j) for i in range(3) for j in range(2)]
```

[(0, 0), (0, 1), (1, 0), (1, 1), (2, 0), (2, 1)]

Flatten nested lists

```
matrix = [[1, 2, 3], [4, 5, 6], [7, 8, 9]]
```

```
flattened = [num for row in matrix for num in row]
```

[1, 2, 3, 4, 5, 6, 7, 8, 9]

Complex List Comprehensions (推導式)

- Real-World Examples

```
# Data processing
```

```
students = [  
    {"name": "Alice", "scores": [85, 92, 78]},  
    {"name": "Bob", "scores": [92, 88, 95]}  
]
```

```
# Calculate averages
```

```
averages = [sum(s["scores"])/len(s["scores"]) for s in students]
```

```
# Filter high performers
```

```
high_performers = [s["name"] for s in students  
                    if sum(s["scores"])/len(s["scores"]) >= 90]
```


Tuple

Tuple Basics

- What are Tuples?
 - Tuples are ordered, Immutable collections
- Key Characteristics
 - Ordered (有序): Items maintain their position
 - Immutable (不可變): Cannot be modified after creation
 - Allow duplicates (允許重複): Same values can appear multiple times
 - Hashable (可雜湊): Can be used as dictionary keys

Tuple Creation

- Creation Methods

Method 1: Parentheses

coordinates = (3, 5)

colors = ("red", "green", "blue")

mixed = ("Python", 42, 3.14, True)

Special cases

empty_tuple = ()

single_item = (42,)

not_tuple = (42)

Empty tuple

Single item (comma required!)

This is just an integer

Method 2: Without parentheses (tuple packing)

point = 10, 20

Creates tuple (10, 20)

person = "Alice", 25, "Engineer"

Method 3: tuple() constructor

list_to_tuple = tuple([1, 2, 3, 4])

(1, 2, 3, 4)

string_to_tuple = tuple("hello")

('h', 'e', 'l', 'l', 'o')

range_to_tuple = tuple(range(5))

(0, 1, 2, 3, 4)

Working with Tuples

- Accessing Elements

```
fruits = ("apple", "banana", "orange", "grape", "kiwi")
```

Indexing (same as lists)

```
first = fruits[0]          # "apple"
```

```
last = fruits[-1]         # "kiwi"
```

Slicing (same as lists)

```
first_three = fruits[:3]   # ('apple', 'banana', 'orange')
```

```
last_two = fruits[-2:]     # ('grape', 'kiwi')
```

Working with Tuples

- Tuple Methods

```
numbers = (1, 2, 3, 2, 4, 2, 5)
```

```
# count() - Count occurrences
```

```
count_2 = numbers.count(2)           # 3
```

```
# index() - Find first occurrence
```

```
pos_3 = numbers.index(3)             # 2
```

```
# Length and membership
```

```
length = len(numbers)               # 7
```

```
has_4 = 4 in numbers                # True
```

Working with Tuples

- Tuple to List Conversion

```
# Convert tuple to list for modification  
original_tuple = (1, 2, 3)  
temp_list = list(original_tuple)  
temp_list.append(4)  
new_tuple = tuple(temp_list) # (1, 2, 3, 4)  
  
# This is the only way to "modify" a tuple!
```

Tuple Unpacking - Extract Values

- Basic Unpacking

```
# Simple unpacking
```

```
point = (10, 20)
```

```
x, y = point
```

```
print(f"x: {x}, y: {y}") # x: 10, y: 20
```

```
# Multiple values
```

```
person = ("Alice", 25, "Engineer")
```

```
name, age, job = person
```

```
print(f"{name} is {age} years old, works as {job}")
```

Tuple Unpacking - Extract Values

- Variable Swapping

```
# Elegant variable swapping  
a = 5  
b = 10  
a, b = b, a           # Swap values!  
print(f'a: {a}, b: {b}') # a: 10, b: 5
```


Tuple Unpacking - Extract Values

- Extended Unpacking

```
numbers = (1, 2, 3, 4, 5, 6, 7, 8, 9)
```

```
# Collect middle elements
```

```
first, *middle, last = numbers
```

```
print(f"First: {first}")
```

```
# 1
```

```
print(f"Middle: {middle}")
```

```
# [2, 3, 4, 5, 6, 7, 8]
```

```
print(f"Last: {last}")
```

```
# 9
```

```
# Collect from beginning
```

```
first, second, *rest = numbers
```

```
print(f"Rest: {rest}")
```

```
# [3, 4, 5, 6, 7, 8, 9]
```

```
# Ignore values with underscore
```

```
_, _, third, *_ = numbers print(f"Third: {third}")
```

```
# 3
```

Returning Multiple Values from Functions

- Multiple Return Values

```
def get_name_age():  
    """Return multiple values as tuple"""  
    return "Alice", 25          # Actually returns ("Alice", 25)  
  
# Unpack the returned tuple  
name, age = get_name_age()  
  
def calculate_stats(numbers):  
    """Calculate multiple statistics"""  
    if not numbers:  
        return 0, 0, 0        # min, max, avg  
    minimum = min(numbers)  
    maximum = max(numbers)  
    average = sum(numbers) / len(numbers)  
    return minimum, maximum, average  
  
# Usage  
data = [85, 92, 78, 96, 89]  
min_score, max_score, avg_score = calculate_stats(data)  
print(f"Min: {min_score}, Max: {max_score}, Avg: {avg_score:.2f}")
```

Returning Multiple Values from Functions

- Advanced Example

```
def analyze_text(text):  
    """Analyze text and return comprehensive stats"""  
    words = text.split()  
    chars = len(text)  
    word_count = len(words)  
    avg_word_length = sum(len(word) for word in words) / word_count  
    if words else 0  
    return chars, word_count, avg_word_length  
  
# Multiple assignment  
char_count, word_count, avg_length = analyze_text("Python is awesome!")  
print(f"Characters: {char_count}, Words: {word_count}, Avg length:  
{avg_length:.2f}")
```

Common Tuple Use Cases

- Coordinates and Points

2D coordinates

origin = (0, 0)

point_a = (3, 4)

point_b = (-2, 5)

3D coordinates

vertex_3d = (1, 2, 3)

RGB colors

red = (255, 0, 0)

green = (0, 255, 0)

blue = (0, 0, 255)

custom_color = (128, 64, 192)

```
def calculate_distance(p1, p2):
```

```
    """Calculate distance between two 2D points"""
```

```
    x1, y1 = p1
```

```
    x2, y2 = p2
```

```
    return ((x2 - x1)**2 + (y2 - y1)**2)**0.5
```

```
distance = calculate_distance((0, 0), (3, 4)) # 5.0
```

Common Tuple Use Cases

- Configuration Data

```
# Database configuration
db_config = ("localhost", 5432, "myapp_db", "admin", "password")
host, port, database, username, password = db_config
# Application settings
app_settings = (
    "MyApp",      # app_name
    "1.0.0",      # version
    True,         # debug_mode
    30            # timeout_seconds
)
# Server endpoints
servers = [
    ("web-server-1", "192.168.1.10", 80),
    ("web-server-2", "192.168.1.11", 80),
    ("db-server", "192.168.1.20", 5432)
]
for name, ip, port in servers:
    print(f"Server: {name} at {ip}:{port}")
```

Named Tuples - Self-Documenting Tuples

- Creating Named Tuples

```
from collections import namedtuple
```

```
# Create named tuple class
```

```
Person = namedtuple('Person', ['name', 'age', 'city'])
```

```
Point = namedtuple('Point', 'x y') # Can also use string format
```

```
# Create instances
```

```
alice = Person("Alice", 25, "New York")
```

```
bob = Person("Bob", 30, "London")
```

```
origin = Point(0, 0)
```

```
endpoint = Point(10, 5)
```

Named Tuples - Self-Documenting Tuples

- Accessing Named Tuple Data

Access by name (more readable)

```
print(f"Name: {alice.name}")
```

```
print(f"Age: {alice.age}")
```

```
print(f"City: {alice.city}")
```

Still works like regular tuple

```
print(f"First element: {alice[0]}")
```

```
name, age, city = alice
```

"Alice"

Unpacking still works

Named Tuples - Self-Documenting Tuples

- Named Tuple Methods

_asdict() - Convert to dictionary

```
alice_dict = alice._asdict()
```

```
print(alice_dict) # {'name': 'Alice', 'age': 25, 'city': 'New York'}
```

_replace() - Create new instance with some fields changed

```
alice_older = alice._replace(age=26)
```

```
print(alice_older) # Person(name='Alice', age=26, city='New York')
```

_fields - Get field names

```
print(Person._fields) # ('name', 'age', 'city')
```


Named Tuples - Self-Documenting Tuples

- Benefits
 - More readable than regular tuples
 - Immutable like tuples
 - Can be used as dictionary keys
 - Self-documenting code

Real-World Tuple Applications

- Database Records

Student records

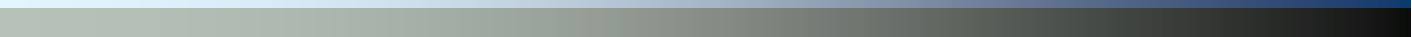
```
students = [  
    (1, "Alice Wang", "Computer Science", 3.8),  
    (2, "Bob Chen", "Mathematics", 3.6),  
    (3, "Charlie Liu", "Physics", 3.9)  
]
```

Process records efficiently

```
for student_id, name, major, gpa in students:  
    status = "Dean's List" if gpa >= 3.8 else "Good Standing"  
    print(f"ID: {student_id}, {name} ({major}), GPA: {gpa}, Status:  
    {status}")
```



Set



Set Basics

- What are Sets ?
 - Sets are unordered, unique collections
- Key Characteristics
 - Unordered (無序): No fixed position for items
 - Unique (唯一): No duplicate values allowed
 - Mutable (可變): Can add/remove items
 - Fast membership testing (快速成員測試): $O(1)$ lookup time

Set Creation

- Creation Methods

Method 1: Curly braces

```
fruits = {"apple", "banana", "orange"}
```

```
numbers = {1, 2, 3, 4, 5}
```

```
mixed = {1, "hello", 3.14, True}
```

Important: Empty set

```
empty_set = set()
```

```
print(type({}))
```

```
print(type(set()))
```

NOT {} (that creates an empty dict!)

<class 'dict'>

<class 'set'>

Method 2: set() constructor

From list (removes duplicates automatically)

```
numbers_list = [1, 2, 2, 3, 3, 3, 4, 5]
```

```
unique_numbers = set(numbers_list) # {1, 2, 3, 4, 5}
```

From string

```
unique_chars = set("hello world") # {'h', 'e', 'l', 'o', ' ', 'w', 'r', 'd'}
```

From tuple

```
set_from_tuple = set((1, 2, 3, 2, 1)) # {1, 2, 3}
```

Essential Set Operations

- Adding Elements

```
colors = {"red", "blue"}
```

```
# add() - Add single element
```

```
colors.add("green")
```

```
print(colors)           # {'red', 'blue', 'green'}
```

```
# Adding duplicate has no effect
```

```
colors.add("red")
```

```
print(colors)           # {'red', 'blue', 'green'} (unchanged)
```

```
# update() - Add multiple elements
```

```
colors.update(["yellow", "purple"])
```

```
colors.update("orange") # Adds each character as separate element
```

```
print(colors)
```

Essential Set Operations

- Removing Elements

```
fruits = {"apple", "banana", "orange"}
```

```
# remove() - Remove element (raises KeyError if not found)
```

```
fruits.remove("banana")
```

```
print(fruits)          # {'apple', 'orange'}
```

```
# discard() - Remove element (no error if not found)
```

```
fruits.discard("grape") # No error even though "grape" not in set
```

```
print(fruits)          # {'apple', 'orange'}
```

```
# pop() - Remove and return arbitrary element
```

```
removed = fruits.pop()
```

```
print(f"Removed: {removed}")
```

```
# clear() - Remove all elements
```

```
fruits.clear()
```

```
print(fruits)          # set()
```

Essential Set Operations

- Membership Testing

```
large_set = set(range(1000000))
```

```
# Very fast membership testing - O(1)
```

```
print(500000 in large_set)      # True - instant!
```

```
print(1500000 not in large_set) # True - instant!
```


Set Mathematical Operations

- Four Basic Operations

```
set_a = {1, 2, 3, 4, 5}
```

```
set_b = {4, 5, 6, 7, 8}
```

Union ($A \cup B$) (聯集) - All elements from both sets

```
union = set_a | set_b # {1, 2, 3, 4, 5, 6, 7, 8}
```

```
union_method = set_a.union(set_b)
```

Same result # Intersection ($A \cap B$) (交集) - Common elements only

```
intersection = set_a & set_b # {4, 5}
```

```
intersection_method = set_a.intersection(set_b)
```

Difference ($A - B$) (差集) - Elements in first but not second

```
difference = set_a - set_b # {1, 2, 3}
```

```
difference_method = set_a.difference(set_b)
```

Symmetric Difference ($A \triangle B$) (對稱差集) - Either but not both

```
sym_diff = set_a ^ set_b # {1, 2, 3, 6, 7, 8}
```

```
sym_diff_method = set_a.symmetric_difference(set_b)
```

Set Mathematical Operations

- Multiple Set Operations

Chain operations

```
set_c = {3, 4, 9, 10}
```

```
result = set_a | set_b & set_c # Intersection has higher precedence
```

```
result2 = (set_a | set_b) & set_c # Use parentheses for clarity
```

Set Relationships and Comparisons

- Subset and Superset Relations

```
set_a = {1, 2, 3}
set_b = {1, 2, 3, 4, 5}
set_c = {1, 2}
# Subset (子集合) - All elements of first set are in second
is_subset = set_a <= set_b          # True (set_a is subset of set_b)
is_subset_method = set_a.issubset(set_b)

# Proper subset (真子集合) - Subset but not equal
is_proper_subset = set_a < set_b    # True
is_proper_subset = set_c < set_a    # True

# Superset (超集合) - Contains all elements of other set
is_superset = set_b >= set_a        # True (set_b is superset of set_a)
is_superset_method = set_b.issuperset(set_a)

# Proper superset (真超集合) - Superset but not equal
is_proper_superset = set_b > set_a  # True
```

Set Relationships and Comparisons

- Disjoint Sets

```
set_x = {1, 2, 3}
```

```
set_y = {4, 5, 6}
```

```
set_z = {3, 4, 5}
```

Disjoint (不相交) - No common elements

```
are_disjoint = set_x.isdisjoint(set_y) # True
```

```
not_disjoint = set_x.isdisjoint(set_z) # False
```

Set Relationships and Comparisons

- Practical Example

```
# User permissions system
```

```
admin_permissions = {"read", "write", "delete", "admin"}
```

```
editor_permissions = {"read", "write"}
```

```
viewer_permissions = {"read"}
```

```
# Check relationships
```

```
print(f"Editor is subset of admin: {editor_permissions <= admin_permissions}")
```

```
print(f"Viewer is subset of editor: {viewer_permissions <= editor_permissions}")
```

```
print(f"Admin and viewer disjoint: {admin_permissions.isdisjoint(viewer_permissions)}")
```

Set Comprehensions and Duplicate Removal

- Set Comprehensions (推導式)

Basic syntax

{expression for item in iterable}

Generate squares

```
squares = {x**2 for x in range(10)} # {0, 1, 4, 9, 16, 25, 36, 49, 64, 81}
```

With conditions

```
even_squares = {x**2 for x in range(10) if x % 2 == 0} # {0, 4, 16, 36, 64}
```

String processing

```
text = "Hello World"
```

```
unique_chars = {char.lower() for char in text if char.isalpha()}
```

```
# {'h', 'e', 'l', 'o', 'w', 'r', 'd'}
```

Set Comprehensions and Duplicate Removal

- Deduplication

```
# Remove duplicates from list while preserving some order
numbers_with_dups = [1, 2, 2, 3, 1, 4, 5, 4, 6]
unique_numbers = list(set(numbers_with_dups)) # Order not preserved
print(unique_numbers) # [1, 2, 3, 4, 5, 6] (order may vary)
# Preserve order while removing duplicates
def remove_duplicates_ordered(sequence):
    seen = set()
    result = []
    for item in sequence:
        if item not in seen:
            seen.add(item)
            result.append(item)
    return result
ordered_unique = remove_duplicates_ordered(numbers_with_dups)
print(ordered_unique) # [1, 2, 3, 4, 5, 6] (order preserved)

# Using dict.fromkeys() for ordered deduplication (Python 3.7+)
ordered_unique2 = list(dict.fromkeys(numbers_with_dups))
```

Set Comprehensions and Duplicate Removal

- Real-World Example

```
# Email list deduplication
```

```
email_list = [  
    "alice@example.com", "bob@example.com", "alice@example.com",  
    "charlie@example.com", "bob@example.com", "diana@example.com"  
]
```

```
unique_emails = list(set(email_list))  
print(f"Original: {len(email_list)} emails")  
print(f"Unique: {len(unique_emails)} emails")
```


Advanced Set Use Cases

- Finding Common Elements

```
# Find common interests among friends
```

```
alice_interests = {"reading", "coding", "music", "travel"}
```

```
bob_interests = {"music", "sports", "coding", "gaming"}
```

```
charlie_interests = {"reading", "music", "art", "travel"}
```

```
# Common to all three
```

```
common_all = alice_interests & bob_interests & charlie_interests
```

```
print(f'All three like: {common_all}') # {'music'}
```

```
# Common to Alice and Bob only
```

```
alice_bob_only = (alice_interests & bob_interests) - charlie_interests
```

```
print(f'Alice and Bob only: {alice_bob_only}')
```

Advanced Set Use Cases

- Data Analysis

Website visitor analysis

```
monday_visitors = {"user1", "user2", "user3", "user4", "user5"}
tuesday_visitors = {"user2", "user4", "user6", "user7", "user8"}
wednesday_visitors = {"user1", "user3", "user5", "user7", "user9"}
```

Returning visitors (visited multiple days)

```
returning_visitors = monday_visitors & tuesday_visitors & wednesday_visitors
print(f"Visited all three days: {returning_visitors}")
```

New visitors each day

```
tuesday_new = tuesday_visitors - monday_visitors
wednesday_new = wednesday_visitors - (monday_visitors | tuesday_visitors)
```

Total unique visitors | 總唯一訪客

```
total_unique = monday_visitors | tuesday_visitors | wednesday_visitors
print(f"Total unique visitors: {len(total_unique)}")
```

Frozen Sets - Immutable Sets

- Creating Frozen Sets

frozenset() constructor

```
immutable_set = frozenset([1, 2, 3, 4, 5])
```

```
colors = frozenset(["red", "green", "blue"])
```

From regular set

```
regular_set = {1, 2, 3}
```

```
frozen = frozenset(regular_set)
```

Empty frozenset

```
empty_frozen = frozenset()
```

```
print(f"Type: {type(immutable_set)}")
```

<class 'frozenset'>

```
print(f"Content: {immutable_set}")
```

frozenset({1, 2, 3, 4, 5})

Frozen Sets - Immutable Sets

- Immutable Characteristics

```
numbers = frozenset([1, 2, 3, 4, 5])
# These operations work (don't modify)
print(1 in numbers) # True
print(len(numbers)) # 5
union = numbers | {6, 7} # Creates new frozenset
intersection = numbers & {3, 4, 5, 6} # Creates new frozenset

# These operations will raise AttributeError
# numbers.add(6)           # AttributeError
# numbers.remove(1)        # AttributeError
# numbers.discard(2)       # AttributeError
```

Frozen Sets - Immutable Sets

- Use as Dictionary Keys

frozensets can be dictionary keys (unlike regular sets)

```
cache = {}
```

```
key1 = frozenset([1, 2, 3])
```

```
key2 = frozenset([4, 5, 6])
```

```
cache[key1] = "First result"
```

```
cache[key2] = "Second result"
```

```
print(cache[frozenset([1, 2, 3])]) # "First result"
```

Dictionary

Dictionary Basics

- What are Dictionaries?
 - Dictionaries are Key-Value Mapping
- Key Characteristics
 - Ordered (有序): Maintains insertion order (Python 3.7+)
 - Mutable (可變): Can be modified after creation
 - Keys must be unique and hashable
 - Fast lookup (快速查找): $O(1)$ average time complexity

Dictionary Creation

- Creation Methods

Method 1: Curly braces with key:value pairs

```
student = {"name": "Alice", "age": 20, "major": "CS"}
numbers = {1: "one", 2: "two", 3: "three"}
mixed = {"string_key": 42, 1: "number_key", (1,2): "tuple_key"}
```

Method 2: dict() constructor

```
person = dict(name="Bob", age=25, city="Taipei")
from_list = dict([("a", 1), ("b", 2), ("c", 3)])
from_zip = dict(zip(["x", "y", "z"], [1, 2, 3]))
```



```
names = ["Alice", "Bob", "Charlie"]
ages = [25, 30, 35]
paired = zip(names, ages)
print(list(paired))
# [('Alice', 25), ('Bob', 30), ('Charlie', 35)]
```

Method 3: Dictionary comprehension

```
squares = {x: x**2 for x in range(5)} # {0: 0, 1: 1, 2: 4, 3: 9, 4: 16}
```

Method 4: fromkeys() class method

```
default_dict = dict.fromkeys(["a", "b", "c"], 0) # {'a': 0, 'b': 0, 'c': 0}
```

Empty dictionary

```
empty_dict = {}
empty_dict2 = dict()
```


Essential Dictionary Operations

- Accessing and Modifying Values

```
student = {"name": "Alice", "age": 20, "major": "CS"}
```

Access values

```
name = student["name"]           # "Alice" (raises KeyError if not found)
```

```
age = student.get("age")         # 20 (returns None if not found)
```

```
grade = student.get("grade", "A") # "A" (default value if not found)
```

Modify values

```
student["age"] = 21              # Update existing key
```

```
student["grade"] = "A"          # Add new key
```

```
student.update({"gpa": 3.8, "year": 2}) # Add multiple keys
```

Remove values

```
del student["year"]             # Remove key (raises KeyError if not found)
```

```
removed_grade = student.pop("grade") # Remove and return value
```

```
last_item = student.popitem()     # Remove and return last (key, value) pair
```

Essential Dictionary Operations

- Dictionary Methods

Get all keys, values, and items

```
keys = student.keys()      # dict_keys(['name', 'age', 'major', 'gpa'])
```

```
values = student.values()  # dict_values(['Alice', 21, 'CS', 3.8])
```

```
items = student.items()    # dict_items([('name', 'Alice'), ('age', 21), ...])
```

Convert to lists if needed

```
key_list = list(student.keys())
```

```
value_list = list(student.values())
```

Check membership

```
has_name = "name" in student      # True (checks keys by default)
```

```
has_alice = "Alice" in student.values()  # True (check in values)
```

Clear all items

```
student.clear()      # {}
```

Dictionary Comprehensions and Merging

- Dictionary Comprehensions

Basic syntax

{key_expression: value_expression for item in iterable}

Create squares dictionary

```
squares = {x: x**2 for x in range(6)} # {0: 0, 1: 1, 2: 4, 3: 9, 4: 16, 5: 25}
```

With conditions

```
even_squares = {x: x**2 for x in range(10) if x % 2 == 0}
```

Transform existing dictionary

```
prices = {"apple": 1.5, "banana": 0.8, "orange": 2.0}
```

```
expensive = {fruit: price for fruit, price in prices.items() if price > 1.0}
```

String manipulation

```
text = "hello world"
```

```
char_count = {char: text.count(char) for char in set(text) if char != ' '}
```

Swap keys and values

```
original = {"a": 1, "b": 2, "c": 3}
```

```
swapped = {value: key for key, value in original.items()} # {1: 'a', 2: 'b', 3: 'c'}
```

Dictionary Comprehensions and Merging

- Dictionary Merging

```
dict1 = {"a": 1, "b": 2}
```

```
dict2 = {"b": 3, "c": 4}
```

```
dict3 = {"d": 5, "e": 6}
```

```
# Method 1: update() method (modifies original)
```

```
result1 = dict1.copy()
```

```
result1.update(dict2) # {"a": 1, "b": 3, "c": 4}
```

```
# Method 2: Dictionary unpacking (Python 3.5+)
```

```
merged = {**dict1, **dict2, **dict3} # {"a": 1, "b": 3, "c": 4, "d": 5, "e": 6}
```

```
# Method 3: Union operator (Python 3.9+)
```

```
# merged_new = dict1 | dict2 | dict3
```

```
# Merge with custom logic
```

```
def merge_with_sum(d1, d2):
```

```
    result = d1.copy()
```

```
    for key, value in d2.items():
```

```
        result[key] = result.get(key, 0) + value
```

```
    return result
```

```
sum_merged = merge_with_sum({"a": 1, "b": 2}, {"b": 3, "c": 4})
```

```
# {"a": 1, "b": 5, "c": 4}
```

Dictionary Comprehensions and Merging

- Dictionary Merging

```
dict1 = {"a": 1, "b": 2}
```

```
dict2 = {"b": 3, "c": 4}
```

```
dict3 = {"d": 5, "e": 6}
```

Merge with list values | 合併列表值

```
def merge_lists(d1, d2):  
    result = d1.copy()  
    for key, value in d2.items():  
        if key in result:  
            result[key].extend(value)  
        else:  
            result[key] = value.copy()  
    return result
```

```
list_dict1 = {"fruits": ["apple", "banana"], "colors": ["red"]}
```

```
list_dict2 = {"fruits": ["orange"], "colors": ["blue", "green"], "numbers": [1, 2]}
```

```
merged_lists = merge_lists(list_dict1, list_dict2)
```

```
# {"fruits": ["apple", "banana", "orange"], "colors": ["red", "blue", "green"], "numbers": [1, 2]}
```

Nested Dictionaries and Complex Data Structures

- Nested Dictionary Structure

Student database with nested information

```
students_db = {  
    "001": {  
        "personal": {  
            "name": "Alice Wang",  
            "age": 20,  
            "email": "alice@example.com"  
        },  
        "academic": {  
            "major": "Computer Science",  
            "year": 2,  
            "gpa": 3.8,  
            "courses": ["Python", "Data Structures", "Algorithms"]  
        },  
        "contact": {  
            "phone": "123-456-7890",  
            "address": {  
                "street": "123 Main St",  
                "city": "Taipei",  
                "country": "Taiwan"  
            }  
        }  
    },  
    "002": {  
        "personal": {  
            "name": "Bob Chen",  
            "age": 21,  
            "email": "bob@example.com"  
        },  
        "academic": {  
            "major": "Mathematics",  
            "year": 3,  
            "gpa": 3.6,  
            "courses": ["Calculus", "Linear Algebra", "Statistics"]  
        }  
    }  
}
```

```
    "002": {  
        "personal": {  
            "name": "Bob Chen",  
            "age": 21,  
            "email": "bob@example.com"  
        },  
        "academic": {  
            "major": "Mathematics",  
            "year": 3,  
            "gpa": 3.6,  
            "courses": ["Calculus", "Linear Algebra", "Statistics"]  
        }  
    }  
}
```

Nested Dictionaries and Complex Data Structures

- Accessing Nested Data

Deep access

```
alice_name = students_db["001"]["personal"]["name"]
```

```
alice_gpa = students_db["001"]["academic"]["gpa"]
```

```
alice_city = students_db["001"]["contact"]["address"]["city"]
```

Safe access with get()

```
def safe_get_nested(data, keys, default=None):
```

```
    for key in keys:
```

```
        if isinstance(data, dict) and key in data:
```

```
            data = data[key]
```

```
        else:
```

```
            return default
```

```
    return data
```

```
bob_phone = safe_get_nested(students_db, ["002", "contact", "phone"], "N/A")
```

```
alice_street = safe_get_nested(students_db, ["001", "contact", "address", "street"])
```

Nested Dictionaries and Complex Data Structures

- Accessing Nested Data

Using try-except for nested access

```
def get_nested_value(data, path):
```

```
    try:
```

```
        result = data
```

```
        for key in path:
```

```
            result = result[key]
```

```
        return result
```

```
    except (KeyError, TypeError):
```

```
        return None
```

```
student_major = get_nested_value(students_db, ["001", "academic", "major"])
```

```
missing_data = get_nested_value(students_db, ["001", "academic", "nonexistent"])
```


Nested Dictionaries and Complex Data Structures

- Modifying Nested Data

Add new course to Alice

```
students_db["001"]["academic"]["courses"].append("Machine Learning")
```

Update nested information

```
students_db["001"]["personal"]["age"] = 21
```

Add new nested structure

```
students_db["002"]["contact"] = {  
    "phone": "987-654-3210",  
    "address": {  
        "street": "456 Second Ave",  
        "city": "Kaohsiung",  
        "country": "Taiwan"  
    }  
}
```

Nested Dictionaries and Complex Data Structures

- Modifying Nested Data

Set nested value function

```
def set_nested_value(data, path, value):
```

```
    current = data
```

```
    for key in path[:-1]:
```

```
        if key not in current:
```

```
            current[key] = {}
```

```
            current = current[key]
```

```
    current[path[-1]] = value
```

Usage example

```
set_nested_value(students_db, ["003", "personal", "name"], "Charlie Lee")
```

```
set_nested_value(students_db, ["003", "academic", "gpa"], 3.9)
```

Nested Dictionaries and Complex Data Structures

- Modifying Nested Data

Flatten nested dictionary

```
def flatten_dict(nested_dict, parent_key="", separator='_'):
    items = []
    for key, value in nested_dict.items():
        new_key = f"{parent_key}{separator}{key}"
        if parent_key else key if isinstance(value, dict):
            items.extend(flatten_dict(value, new_key, separator).items())
        else:
            items.append((new_key, value))
    return dict(items)
```

```
alice_flat = flatten_dict(students_db["001"])
```

```
# {'personal_name': 'Alice Wang', 'personal_age': 21, 'academic_major':  
'Computer Science', ...}
```

Advanced Dictionary Methods and Patterns

- Advanced Dictionary Methods

```
# setdefault() - Get or set default value
word_count = {}
text = "hello world hello python"
for word in text.split():
    word_count.setdefault(word, 0)
    word_count[word] += 1

# Result: {'hello': 2, 'world': 1, 'python': 1}

# Alternative using get()
word_count2 = {}
for word in text.split():
    word_count2[word] = word_count2.get(word, 0) + 1

# setdefault with lists
groups = {}
students = [("Alice", "CS"), ("Bob", "Math"), ("Charlie", "CS")]
for name, major in students:
    groups.setdefault(major, []).append(name)

# Result: {'CS': ['Alice', 'Charlie'], 'Math': ['Bob']}
```

Advanced Dictionary Methods and Patterns

- Practical Dictionary Patterns

Grouping data

```
students = [  
    ("Alice", "CS"), ("Bob", "Math"), ("Charlie", "CS"),  
    ("Diana", "Physics"), ("Eve", "Math")  
]
```

Group by major

```
by_major = {}  
for name, major in students:  
    if major not in by_major:  
        by_major[major] = []  
    by_major[major].append(name)
```

Using setdefault()

```
by_major2 = {}  
for name, major in students:  
    by_major2.setdefault(major, []).append(name)  
print(by_major2) # {'CS': ['Alice', 'Charlie'], 'Math': ['Bob', 'Eve'], 'Physics': ['Diana']}
```

Advanced Dictionary Methods and Patterns

- Practical Dictionary Patterns

Inverting dictionary

```
original = {"a": 1, "b": 2, "c": 1, "d": 3}
```

```
inverted = {}
```

```
for key, value in original.items():
```

```
    inverted.setdefault(value, []).append(key)
```

Result: {1: ['a', 'c'], 2: ['b'], 3: ['d']}

Advanced Dictionary Methods and Patterns

- Practical Dictionary Patterns

Dictionary as a switch statement

```
def process_grade(letter_grade):  
    grade_map = {  
        "A": (4.0, "Excellent"),  
        "B": (3.0, "Good"),  
        "C": (2.0, "Average"),  
        "D": (1.0, "Below Average"),  
        "F": (0.0, "Fail")  
    }  
    gpa, description = grade_map.get(letter_grade, (0.0, "Invalid Grade"))  
    return {"gpa": gpa, "description": description}
```

```
result = process_grade("A") # {'gpa': 4.0, 'description': 'Excellent'}
```

Time Complexity Comparison

Operation	List 串列	Tuple 字元組	Set 集合	Dict 字典	Notes
Access	$O(1)$	$O(1)$	N/A	Avg. $O(1)$ Worst $O(n)$	透過索引/ 鍵存取
Search	$O(n)$	$O(n)$	Avg. $O(1)$ Worst $O(n)$	Avg. $O(1)$ Worst $O(n)$	in運算子
Insert	$O(1) \sim O(n)$	N/A	Avg. $O(1)$ Worst $O(n)$	Avg. $O(1)$ Worst $O(n)$	位置相關
Delete	$O(1) \sim O(n)$	N/A	Avg. $O(1)$ Worst $O(n)$	Avg. $O(1)$ Worst $O(n)$	位置相關
Length	$O(1)$	$O(1)$	$O(1)$	$O(1)$	len()函數

String Processing

String Indexing and Slicing

- String Indexing

```
# Positive indexing
text = "Python"
print(text[0])    # 'P' - First character
print(text[5])    # 'n' - Last character

# Negative indexing
print(text[-1])   # 'n' - First from the end
print(text[-6])   # 'P' - Sixth from the end

# Index position diagram
# P y t h o n
# 0 1 2 3 4 5 (Positive indexing)
# -6 -5 -4 -3 -2 -1 (Negative indexing)
```

- Key Points

- Strings are immutable
- Indexing starts from 0
- Negative indexing starts from -1 (last character)
- Out of range index causes IndexError

String Slicing in Detail

```
string[start:stop:step]
```

- Examples and Applications

```
text = "Python Programming"
```

```
# Basic slicing
```

```
print(text[0:6]) # 'Python' - Index 0 to 5
```

```
print(text[7:]) # 'Programming' - From index 7 to end
```

```
print(text[:6]) # 'Python' - From start to index 5
```

```
print(text[:]) # 'Python Programming' - Complete string
```

```
# Step slicing
```

```
print(text[::2]) # 'Pto rgamn' - Every other character
```

```
print(text[1::2]) # 'yhnPoarmig' - From index 1, every other
```

```
print(text[::-1]) # 'gnimmargorP nohtyP' - Reverse string
```

String Slicing in Detail

Practical examples

```
email = "user@example.com"
```

```
username = email[:email.index('@')]          # 'user'
```

```
domain = email[email.index('@')+1:]          # 'example.com'
```

Get file extension

```
filename = "document.pdf"
```

```
extension = filename[filename.rfind('.')+1:] # 'pdf'
```

- Use Cases
 - Extract substrings
 - Reverse strings
 - Data parsing
 - Format processing

String Methods - Conversion

- Case Conversion

```
text = "Python Programming"

# Case conversion methods
print(text.upper())          # 'PYTHON PROGRAMMING' - All uppercase
print(text.lower())          # 'python programming' - All lowercase
print(text.capitalize())     # 'Python programming' - First letter capitalized
print(text.title())          # 'Python Programming' - Title case
print(text.swapcase())       # 'pYTHON pROGRAMMING' - Swap case

# Practical example
name = "john doe"
print(name.title())          # 'John Doe' - Standardize name format

# Case-insensitive comparison
password = "PyThOn123" if password.lower() == "python123":
    print("Password matches!")
```

String Methods - Conversion

- Common Use Cases
 - Data standardization
 - User input processing
 - Case-insensitive comparison
 - Text formatting

String Methods - Searching

- Search and Check Methods

```
text = "Python is awesome, Python is powerful"
```

Finding substrings

```
print(text.find("Python"))      # 0 - First occurrence position
print(text.find("Python", 10))  # 19 - Search from index 10
print(text.find("Java"))        # -1 - Returns -1 if not found
print(text.index("Python"))     # 0 - Similar to find, but raises error if not found
# print(text.index("Java"))     # ValueError!
```

Counting occurrences

```
print(text.count("Python"))     # 2 - "Python" appears 2 times
print(text.count("is"))         # 2 - "is" appears 2 times
```

String Methods - Searching

- Search and Check Methods

Checking start and end

```
email = "user@example.com"
```

```
print(email.startswith("user"))    # True
```

```
print(email.endswith(".com"))      # True
```

```
print(email.endswith(".org"))      # False
```

Practical example

```
url = "https://www.example.com"
```

```
if url.startswith("https://"):
```

```
    print("Secure connection")
```

```
elif url.startswith("http://"):
```

```
    print("Unsecure connection")
```

- Applications
 - Text search
 - Data validation: URL validation, File type checking, ...

String Methods - Modification

- Modification and Cleaning Methods

Replace

```
text = "Hello World, Hello Python"
```

```
print(text.replace("Hello", "Hi"))           # 'Hi World, Hi Python'
```

```
print(text.replace("Hello", "Hi", 1))        # 'Hi World, Hello Python' - Replace only once
```

Strip whitespace

```
messy = " Python "
```

```
print(messy.strip())                        # 'Python' - Remove whitespace from both ends
```

```
print(messy.lstrip())                      # 'Python ' - Remove left whitespace
```

```
print(messy.rstrip())                      # ' Python' - Remove right whitespace
```

Strip specific characters

```
url = "https://www.example.com/"
```

```
print(url.strip("https://"))              # 'www.example.com/'
```

```
print(url.rstrip("/"))                    # 'https://www.example.com'
```

String Methods - Modification

- Split

Split

```
sentence = "Python is awesome"
```

```
words = sentence.split() # ['Python', 'is', 'awesome']
```

```
print(words)
```

```
csv_data = "John,25,Engineer"
```

```
data = csv_data.split(",") # ['John', '25', 'Engineer']
```

```
print(data)
```

```
path = "home/user/documents/file.txt"
```

```
folders = path.split("/") # ['home', 'user', 'documents', 'file.txt']
```

```
print(folders)
```

String Methods - Modification

- Join

```
# Join
```

```
words = ["Python", "is", "awesome"]
```

```
sentence = " ".join(words) # 'Python is awesome'
```

```
print(sentence)
```

```
csv_line = ",".join(["John", "25", "Engineer"]) # 'John,25,Engineer'
```

```
print(csv_line)
```

```
path = "/" .join(["home", "user", "documents"]) # 'home/user/documents'
```

```
print(path)
```

String Methods - Modification

- Practical Examples

Data cleaning

```
user_input = " john.doe@email.com "  
clean_email = user_input.strip().lower()  
print(clean_email)
```

'john.doe@email.com'

CSV processing

```
csv_line = "Alice,30,Designer,New York"  
name, age, job, city = csv_line.split(",")  
print(f"Name: {name}, Age: {age}")
```

Path manipulation

```
base_path = "/var/www"  
subdirs = ["html", "images", "css"]  
full_path = "/" .join([base_path] + subdirs)  
print(full_path)
```

'/var/www/html/images/css'

String Formatting - Old Style

- % Formatting

Basic formatting

```
name = "Alice"
```

```
age = 25
```

```
print("Name: %s, Age: %d" % (name, age))
```

Output: Name: Alice, Age: 25

Format specifiers

```
print("Integer: %d" % 42)
```

Integer

```
print("Float: %f" % 3.14159)
```

Float

```
print("Float (2 decimals): %.2f" % 3.14159)
```

3.14

```
print("String: %s" % "Hello")
```

String

```
print("Hex: %x" % 255)
```

ff (hexadecimal)

```
print("Octal: %o" % 8)
```

10 (octal)

String Formatting - Old Style

Width and alignment

```
print("Right aligned: %10s" % "Python")
```

' Python'

```
print("Left aligned: %-10s" % "Python")
```

'Python '

```
print("Zero padded: %05d" % 42)
```

'00042'

Multiple values

```
print("%s scored %d points in %s" % ("Alice", 95, "Math"))
```

*# Output: Alice scored 95
points in Math*

- Notes
 - This is Python 2 style formatting
 - Still works but not recommended for new projects
 - Syntax similar to C's printf

String Formatting - format() Method

- str.format() Method

Basic usage

```
print("Hello, {}".format("World"))
```

Output: Hello, World!

Positional arguments

```
print("{0} + {1} = {2}".format(5, 3, 8))
```

Output: 5 + 3 = 8

```
print("{2} {1} {0}".format("a", "b", "c"))
```

Output: c b a

Named arguments

```
print("Hello, {name}! You are {age} years old.".format(name="Alice", age=25))
```

Output: Hello, Alice! You are 25 years old.

Mixed arguments

```
print("{0}, your score is {score}".format("Bob", score=95))
```

Output: Bob, your score is 95

String Formatting - format() Method

Accessing attributes

```
class Person:
```

```
    def __init__(self, name, age):
        self.name = name
        self.age = age
```

```
person = Person("Alice", 25)
```

```
print("Name: {0.name}, Age: {0.age}".format(person)) # Output: Name: Alice, Age: 25
```

Number formatting

```
print("Float: {:.2f}".format(3.14159))           # 3.14
print("Percentage: {:.1%}".format(0.85))         # 85.0%
print("Scientific: {:.2e}".format(1000))          # 1.00e+03
print("Binary: {:b}".format(10))                  # 1010
print("Hex: {:x}".format(255))                     # ff
```

Alignment and padding

```
print("Left: {:<10}".format("Python"))           # 'Python '
print("Right: {:>10}".format("Python"))          # ' Python'
print("Center: {:^10}".format("Python"))         # ' Python '
print("Fill: {:*^10}".format("Python"))          # '**Python**'
```


String Formatting - f-strings [Recommended]

- f-string (Formatted String Literals) - Python 3.6+

```
# Basic f-string
name = "Alice"
age = 25
print(f"Hello, {name}! You are {age} years old.") # Output: Hello, Alice! You are 25 years old.

# Expressions inside f-strings
x = 10
y = 20
print(f"{x} + {y} = {x + y}") # Output: 10 + 20 = 30

# Calling functions
def greet(name):
    return f"Hello, {name}"

print(f"Message: {greet('Bob')}") # Output: Message: Hello, Bob
```

String Formatting - f-strings [Recommended]

Number formatting

```
pi = 3.14159265359
```

```
print(f"Pi: {pi:.2f}")           # 3.14
```

```
print(f"Pi: {pi:.4f}")           # 3.1416
```

```
price = 1234.5
```

```
print(f"Price: ${price:,.2f}")    # $1,234.50
```

```
percentage = 0.856
```

```
print(f"Success rate: {percentage:.1%}")    # 85.6%
```

Alignment and width

```
word = "Python"
```

```
print(f"Left: {word:<10}")         # 'Python  '
```

```
print(f"Right: {word:>10}")        # '  Python'
```

```
print(f"Center: {word:^10}")       # ' Python '
```

```
print(f"Fill: {word:*^10}")        # '**Python**'
```

String Formatting - f-strings [Recommended]

```
# Date formatting
from datetime import datetime
now = datetime.now()
print(f"Date: {now:%Y-%m-%d}")           # 2024-03-15
print(f"Time: {now:%H:%M:%S}")           # 14:30:45
print(f"Full: {now:%Y-%m-%d %H:%M:%S}")  # 2024-03-15 14:30:45

# Debug mode (Python 3.8+)
x = 42
print(f"{x=}") # x=42
print(f"{x * 2=}") # x * 2=84
```

- Why f-strings?
 - More concise and readable
 - Faster execution speed
 - Supports expressions
 - Convenient debug mode

String Formatting - f-strings [Recommended]

- Practical Examples

Table formatting

```
products = [  
    ("Apple", 1.2, 10),  
    ("Banana", 0.5, 25),  
    ("Orange", 1.5, 15)  
]  
  
print(f"{'Product':<10} {'Price':>8} {'Quantity':>10}")  
print("-" * 30)  
for name, price, qty in products:  
    print(f"{'name':<10} ${price:>7.2f} {qty:>10}")
```

Output:

```
# Product Price      Quantity  
# -----  
# Apple   $ 1.20      10  
# Banana  $ 0.50      25  
# Orange  $ 1.50      15
```


String Formatting - f-strings [Recommended]

- Practical Examples

```
# Multi-line f-strings
name = "Alice"
age = 25
city = "Taipei"
message = f"""
User Profile:
    Name: {name}
    Age: {age}
    City: {city}
    Status: {'Active' if age >= 18 else 'Inactive'}
"""
print(message)
```

String Checking Methods

- Character Type Checking

Alphabetic checks

```
print("Python".isalpha()) # True - All letters
print("Python3".isalpha()) # False - Contains digit
print("").isalpha() # False - Empty string
```

Numeric checks

```
print("12345".isdigit()) # True - All digits
print("123.45".isdigit()) # False - Contains decimal point
print("").isdigit() # False - Empty string
```

Alphanumeric checks

```
print("Python3".isalnum()) # True - Letters or digits
print("Python 3".isalnum()) # False - Contains space
print("Python-3".isalnum()) # False - Contains special char
```

Case checks

```
print("PYTHON".isupper()) # True - All uppercase
print("Python".isupper()) # False - Has lowercase
print("python".islower()) # True - All lowercase
print("Python".islower()) # False - Has uppercase
```

String Checking Methods

Whitespace checks

```
print(" ".isspace())
```

True - All whitespace

```
print("").isspace())
```

False - Empty string

```
print("Hello World".isspace())
```

False - Contains non-whitespace

Title case checks

```
print("Python Programming".istitle())
```

True

```
print("Python programming".istitle())
```

False

```
print("PYTHON PROGRAMMING".istitle())
```

False

Decimal checks

```
print("123".isdecimal())
```

True

```
print("1½".isdecimal())
```

False

```
print("123.45".isdecimal())
```

False

Numeric checks (broader)

```
print("123".isnumeric())
```

True

```
print("1½".isnumeric())
```

True - Unicode fraction Unicode

```
print("IV".isnumeric())
```

True - Roman numeral

String Checking Methods

- Practical Applications

Input validation

```
def validate_username(username):  
    """  
    Validate username:  
    - Only letters and numbers  
    - Length 3-20 characters  
    """  
    if not username.isalnum():  
        return False, "Username can only contain letters and numbers"  
    if not (3 <= len(username) <= 20):  
        return False, "Username must be 3-20 characters"  
    return True, "Valid username"
```

Test

```
print(validate_username("user123"))    # (True, 'Valid username')  
print(validate_username("user@123"))  # (False, 'Username can only contain...')  
print(validate_username("ab"))         # (False, 'Username must be 3-20...')
```

String Checking Methods

- Practical Applications

Password strength checker

```
def check_password_strength(password):  
    """Check password strength"""  
    has_upper = any(c.isupper() for c in password)  
    has_lower = any(c.islower() for c in password)  
    has_digit = any(c.isdigit() for c in password)  
    has_special = not password.isalnum()  
  
    strength = sum([has_upper, has_lower, has_digit, has_special])  
  
    if strength == 4 and len(password) >= 8:  
        return "Strong"  
    elif strength >= 3 and len(password) >= 6:  
        return "Medium"  
    else:  
        return "Weak"  
  
print(check_password_strength("Pass123!"))  
print(check_password_strength("password"))  
print(check_password_strength("Pass123"))
```

*# Strong
Weak
Medium*

String Checking Methods

- Practical Applications

Data type detection

```
def detect_data_type(value):  
    """Detect data type"""  
    if value.isdigit():  
        return "Integer"  
  
    try:  
        float(value)  
        return "Float"  
    except ValueError:  
        pass  
  
    if value.lower() in ['true', 'false']:  
        return "Boolean"  
    return "String"  
  
print(detect_data_type("123")) # Integer  
print(detect_data_type("123.45")) # Float  
print(detect_data_type("true")) # Boolean  
print(detect_data_type("hello")) # String
```

Regular Expressions

Regular Expressions Basics

- What are Regular Expressions?
 - Regular expressions (regex) are powerful text pattern matching tools for searching, validating, and processing text.
- Import re Module

```
import re
```

Basic Patterns

Special characters

. # Match any single character
(except newline)
^ # Match start of string
\$ # Match end of string
*** # Match 0 or more occurrences
+ # Match 1 or more occurrences
? # Match 0 or 1 occurrence
{n} # Match exactly n occurrences
{n,} # Match at least n occurrences
{n,m} # Match n to m occurrences

Character classes

[abc] # Match a, b, or c
[a-z] # Match any lowercase letter
[A-Z] # Match any uppercase letter
[0-9] # Match any digit
[^abc] # Match any character except a,
b, c

Predefined character classes

\d # Match digit [0-9]
\D # Match non-digit [^0-9]
\w # Match word character [a-zA-Z0-9_]
\W # Match non-word character
\s # Match whitespace (space, tab, newline)
\S # Match non-whitespace

Basic Examples

```
import re

# Simple matching
text = "The phone number is 123-456-7890"

# Find digits
digits = re.findall(r'\d', text)
print(digits) # ['1', '2', '3', '4', '5', '6', '7', '8', '9', '0']

# Find sequences of digits
numbers = re.findall(r'\d+', text)
print(numbers) # ['123', '456', '7890']

# Find words
words = re.findall(r'\w+', text)
print(words) # ['The', 'phone', 'number', 'is', '123', '456', '7890']

# Match phone pattern
phone = re.findall(r'\d{3}-\d{3}-\d{4}', text)
print(phone) # ['123-456-7890']
```

Regular Expression Functions

- Main Functions of re Module

```
import re

text = "Contact us at info@example.com or support@example.org"

# 1. re.search() - Search for first match
match = re.search(r'\w+@\w+\.\w+', text)
if match:
    print(f"Found: {match.group()}")           # info@example.com
    print(f"Position: {match.start()}-{match.end()}") # 14-31

# 2. re.match() - Match from beginning
text2 = "Python is awesome"
match = re.match(r'Python', text2)
if match:
    print(f"Matched: {match.group()}")         # Python
match = re.match(r'awesome', text2)
if match:
    print("Matched")
else:
    print("No match")                         # No match
```


Regular Expression Functions

3. re.findall() - Find all matches

```
emails = re.findall(r'\w+@\w+\.\w+', text)
print(emails) # ['info@example.com', 'support@example.org']
```

4. re.finditer() - Iterator of match objects

```
for match in re.finditer(r'\w+@\w+\.\w+', text):
    print(f"Email: {match.group()} at position {match.start()}")
```

5. re.sub() - Replace matches

```
censored = re.sub(r'\w+@\w+\.\w+', '[EMAIL REDACTED]', text)
print(censored)    # Output: Contact us at [EMAIL REDACTED] or [EMAIL REDACTED]
```

With function replacement

```
def mask_email(match):
    email = match.group()
    name, domain = email.split('@')

    return f"{name[0]}***@{domain}"
masked = re.sub(r'\w+@\w+\.\w+', mask_email, text)
print(masked)    # Output: Contact us at i***@example.com or s***@example.org
```

Regular Expression Functions

6. re.split() - Split by pattern

```
text3 = "apple,banana;orange:grape"
```

```
fruits = re.split(r'[,:;]', text3)
```

```
print(fruits)          # ['apple', 'banana', 'orange', 'grape']
```

7. re.compile() - Compile pattern for reuse

```
email_pattern = re.compile(r'\w+@\w+\.\w+')
```

```
emails = email_pattern.findall(text)
```

```
print(emails)          # ['info@example.com', 'support@example.org']
```